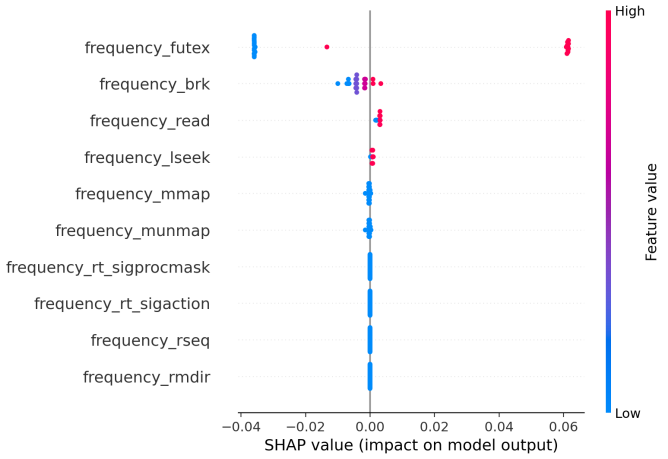


Fig. 6: Top-10 important features that explain DYNAHUG's predictions on benign models

XI. APPENDIX

Algorithm 1 DYNAHUG Algorithm

Input: TAG, modelToAnalyze
Output: Malicious or Benign
1: **// Phase 1: Crawling**
2: $M_{\text{tag}} \leftarrow$ fetchModels("$PyTorch$", TAG)
3: **// Phase 2: Dynamic Analysis**
4: traces $\leftarrow \emptyset$
5: **for** $M_i \in M_{\text{tag}}$ **do** ▷ Model deserialization
6: executeModelInSandbox(M_i)
7: traces $\leftarrow$ traces $\cup$ getTraces(M_i)
8: **end for**
9: **// Phase 3: Model Training**
10: $X \leftarrow \emptyset$ ▷ Training dataset
11: **for** trace$_i$ $\in$ traces **do**
12: $d_i \leftarrow$ dataProcessing(trace$_i$)
13: $X \leftarrow X \cup$ featureEngineering(d_i)
14: **end for**
15: $\mathcal{M} \leftarrow \emptyset$ ▷ ML Classifier evaluation
16: modelArch $\leftarrow$ OneClassSVM
17: hyperparams $\leftarrow \{kernel : [\texttt{RBF}, \texttt{linear}, ...], ...\}$
18: **for** $h \in$ hyperparams **do** ▷ Grid Search CV
19: $\mathcal{M} \leftarrow \mathcal{M} \cup$ train(X, modelArch, h)
20: **end for**
21: $(m_{\text{best}}, h_{\text{best}}) \leftarrow$ evaluation($\mathcal{M}$)

A. DYNAHUG *Overview*

DYNAHUG's steps are color mapped in the algorithm (Algorithm 1) and the workflow diagram (Figure 2): *Crawling* steps are highlighted in orange, *Dynamic Analysis* steps in blue and *Model Training* steps in green.

The goal of DYNAHUG is to learn the benign behavior of PTMs for a specific ML task (e.g., text generation). It achieves this via dynamic analysis and machine learning. Given a PTM under test (PUT) and the task tag of the PTM (e.g., text generation), DYNAHUG learns the benign behavior of benign models in the task tag and identifies the PUT to be either *malicious* or *benign*.

To achieve this, DYNAHUG first fetches benign PTMs for the task at hand from a Model Hub (e.g., Hugging Face) by filtering for the task tag and the model format (e.g, Pickle) (Algorithm 1:line 2). Once a large number (2K) of benign PTMs are retrieved, DYNAHUG loads (deserialises) each model in a sandbox and collects its system call traces using runtime observability tools like `strace` (Algorithm 1:line 5,6). It then processes the collected traces into meaningful features, transforming them into a structured and interpretable format (Algorithm 1:line 11). Once the traces are processed, it engineers new features from the existing data, i.e., binary and frequency features (Algorithm 1:line 12). Next, DYNAHUG learns an ML model detector by first tuning the model to maximize performance by passing this processed data for training alongside the hyperparameters and ML architectures (Algorithm 1:line 15,16,19). Using the F1-score as the main evaluation metric, the best performing model is then chosen as DYNAHUG. Finally, at inference, DYNAHUG detects whether the *PUT* is benign or malicious. It first collects and processes the trace data of the PUT (similarly to the *Model training* step), then passes the processed data for a detection output of *malicious* or *benign*.

Example 2: MALHUG payload [75]

```
2: c    GLOBAL     '__builtin__ eval'
20: q    BINPUT     0
22: X    BINUNICODE
  'exec(\'\'\'\nprint("Hello, I am
  'Gorden
  Ramsey!")\n\'\'\') or dict()'
```

Example 3: PyPI payload

```
2: c    GLOBAL     'raft run'
12: q    BINPUT     0
14: X    BINUNICODE "zsh -c
  'zmodload
 zsh/net/tcp
 && ztcp 127.0.0.1 4444
   && zsh >\&$REPLY 2>&$REPLY
  0>\&$REPLY'"
```

B. *Dataset collection and crawling setup*

DYNAHUG utilizes functions from the `huggingface_hub` [45] library to fetch a list of repository names from HF to crawl. We pass *likes* and *pipeline tag* as arguments to this function to obtain a list of repositories ranked in descending order of *likes* and clustered based on the *pipeline tag* in HF. By iterating through each repository in the list, we obtain metadata information such as the type of files being stored. We look through the list of files at the top level of the repository and check for the presence of `pytorch_model.bin`. For each PyTorch model file, we check its HF security status and skip it if the model file was identified by HF as "unsafe". We retrieve the size of the file and estimate the memory usage when it is deserialized during dynamic analysis to avoid Out Of Memory (OOM) issues. If the memory safety check succeeds, the `pytorch_model.bin` file is downloaded from the repository and we check that it contains a Pickle file.[3] If all the aforementioned conditions are met, we store any relevant metadata about the repository

[3]This step is required since `.bin` files do not guarantee the presence of `.pickle` or `.pkl` files.

in a CSV file, i.e., number of likes, downloads, date of last commit and HF security tool [17], [19], [18] detection.

C. Baseline Selection

Security Scanners: We chose PickleScan [20] and Modelscan [21] as they are the closest open-source alternatives to the currently used scanners on Hugging Face [18], [17]. PickleScan is the base version of Hugging Face's HF PickleScan, while Modelscan is developed by the same company, ProtectAI, which also developed Hugging Face's Guardian. We also use Fickling [24], developed by Trail of Bits, as it provides novel detection methods. To the best of our knowledge, ModelTracer [23] is the only open-source dynamic scanner for malicious PTM detection.

LLM baseline: We employ Llama-3.1-8B-Instruct-tuned (Llama-3.1) for the LLM baseline since the Meta-Llama series of LLMs is widely recognized as one of the state-of-the-art open source models and it has demonstrated strong capabilities in security-related applications [76], [77], [78]. It is free to use, computationally efficient and instruction-tuned to produce standardized outputs. GPT-5.2 was also used as a baseline due to its strong performance amongst the current state-of-the-art LLMs.

Excluded Scanners: For **RQ2**, we exclude the following types of scanners because they are not applicable to our threat model, impractical in practice, or impossible to execute on our dataset:
a.) Hugging Face Scanners: We note that Hugging Face (HF) runs closed-source versions of security scanners on uploaded models. However, we do not evaluate against the HF scanners as we already use them to filter for models that are benign and thus would be inhenerently biased. We also cannot feasibly evaluate the HF Scanners against our injected malicious set as the only way to run them would be to upload them to HF –calling for the need of upload of 6000 injected malicious models– which comes with a risk of violating the platform's policies.
b.) Restricted Loading Environments (RLEs): We determined that RLEs were not applicable to our threat model, as they require a security-aware user. Specifically, we note that Weights-Only Unpickler [26], developed by PyTorch, can be bypassed by merely instructing the user to set `weights_only=False` while loading the model. PickleBall [25] is also a RLE that overrides PyTorch's Weights-Only Unpickler[26]. Furthermore, PickleBall requires manual specification of security policies for each model, which requires significant time and effort. It does not scale to our setting where we scan thousands of PTMs (over 39K) or generalise to model hub security scenarios (e.g., Hugging Face containing 2.2M models [79]).

D. Baseline setting

We compare DYNAHUG to the open-source state-of-the-art (SOTA) malicious PTM detectors. In particular, three static detectors (`PickleScan` [20], `Modelscan` [21], `Fickling` [24]) and one dynamic detector (`ModelTracer` [23]). We also compare DYNAHUG to open-weights and closed-source SOTA LLMs, namely Llama-3.1 [64] and GPT-5.2 [65].

Baseline Scanner setup: We execute PickleScan and ModelScan via a Python script that calls a shell command to run the tool, given the path to the PUT. We employ `analyze_pickle_safety`, to obtain various security threat levels (such as `LIKELY_UNSAFE, OVERTLY_MALICIOUS`) for Fickling. To run ModelTracer, we execute the code provided by its GitHub repository [80].

LLM baseline setup: For the LLM baseline evaluation, GPT-5.2 and Llama-3.1 were provided raw traces obtained from dynamic analysis as its input. This design choice was made since LLMs are known to have a strong capability to develop a rich semantic understanding of text, enabling it to have a deeper insight into the relationship between the different system calls which occur within an `strace` [52] log file. Due to the small context window of Llama-3.1-8B-Instruct-tuned [64] (i.e., maximum 128k tokens [73]) and the limited CUDA memory of the NVIDIA A100 chip (i.e., 64GB CUDA memory [63]), passing in the `strace` logs in its entirety would leave the program susceptible to either CUDA OOM error or the LLM losing out on vital information for accurate detections. To mitigate this, we filter out which system calls are needed to be passed for the LLMs to have an understanding of the overall functioning of the deserialized PyTorch file. We curated a list of 35 system calls borrowed from *Brown et al.* [81] alongwith an additional 23 system calls to filter the `strace` logs. The same filtered `strace` logs were provided to GPT-5.2 to maintain similar settings with Llama-3.1. The LLMs was instructed with Few-Shot prompting [82] to use this filtered `strace` log to look for any red flags during pickle deserialization. Example 4 portrays the system prompt and a sample LLM response. To mitigate randomness, the `temperature` of the LLMs was set to zero (0).

E. Ablation, Probing and Sensitivity Setting

Feature set: All feature sets displayed in Figure 4 were used to study the effect of each feature on the model performance. In this study, the model complexity was varied while keeping the rest of the hyperparameter settings the same as the DYNAHUG (default). The *presence* only model was trained on features representing the presence of individual system calls. The *frequency* only model was trained on frequency of individual system calls. The *presence and frequency* model (DYNAHUG (default)) was trained on a combination of *presence* and *frequency* features. Subsequently, *Sequence* and *Process Sequence* features were concatenated to train the respective *presence, frequency, sequence* and *presence, frequency, sequence, process sequence* models.

Architecture: *OCSVM*, *SGDSVM* and *IF* were trained with their best hyperparameter setting.

Analysis Type: The *dynamic* model was trained with the workflow showcased in Figure 2. The *static* model processed static opcodes from disassembled models instead of `strace` log data while following the same workflow as the *dynamic* model. The *hybrid* model was trained with the combined feature set of *static* and *dynamic* models.

TABLE X: DYNAHUG's performance of sequence features on varying n-grams

	Benign HF (2025)	Malicious Real (25)	Malicious Injected (2000)	TP	TN	FP	FN	Precision	Recall	F1-score
DYNAHUG (2-gram)	51	**24**	**2000**	**2024**	1974	51	**1**	**0.9754**	**0.9995**	**0.9873**
3-gram	36	**24**	1922	1946	1989	36	79	0.9818	0.9610	0.9713
4-gram	**27**	**24**	1901	1925	**1998**	**27**	100	0.9861	0.9506	0.9681

Clustering: To test whether clustering improves performance, we trained a model without task-based filtering. We then compare this model against DYNAHUG (default).

Dataset Size Sensitivity: For sensitivity analysis (dataset sizes), we trained our model on the data retrieved from the top `1000`, `2000` (DYNAHUG (default)) and `3000` repositories, sorted by likes.

Example 4: System prompt and LLM response

```
Prompt:
You are analyzing strace logs to detect malicious activity in PyTorch
     files.

Look for RED FLAGS like these:
- Process creation (execve,fork,clone,...)
- Network connections (socket, connect, bind, ...)
- File writes outside normal directories
- Access to sensitive files (/etc/passwd, SSH keys,...)
- Essentially unusual system calls
during deserialization

IMPORTANT: You must respond in EXACTLY this format:
Answer: Malicious
Reason: Brief explanation
...
Example 1:
Log shows: execve("/bin/bash", ["/bin/bash", "-c", "rm -rf /"])
Answer: Malicious
Reason: Executes destructive shell command during deserialization
...
--------------------------------------------------------------------------
LLM Response:
Answer: Malicious
Reason: The process attempts to connect to a suspicious IP address
     (192.248.1.167) on port 4242, which could be a sign of a
     malicious activity.
```

F. Data Preprocessing

Once the hyperparameter combination is selected, we utilize the `DictVectorizer` from `scikit-learn` [56] to transform the feature-value mapping to a matrix which is comprehensible to the classifier. The values within this matrix are normalized by applying the `StandardScaler(nomean=True)` class. The centralization of the values by the mean is disabled to prevent the data from losing its sparse nature and optimizing memory usage by avoiding the sparse matrix from becoming a dense matrix. This normalized data is fitted to the classifier of choice and this process is repeated until a classifier with the highest possible average F1-score on the validation set is selected as the final model. The final model is also evaluated on the test set to gain deeper insights of the model's performance on unseen data.

G. Hyperparameter Choices

For OCSVM, we tune parameters *nu* (`0.01` to `0.5`), *kernel* (`linear`, `RBF` and `sigmoid`), and *gamma* (`0.01`, `0.1`, `1` and `auto`). For SGDSVM, we tune batch_size (`32`, `64`, `128` and `1000`). Finally, for IF, we tune *n_estimators* (`50` to `150`), *contamination_rate* (`0.001`, `0.01`, `0.1` and `auto`) and *max_samples* (`128`, `256`, `512` and `1000`).

H. Metrics and Measures

DYNAHUG uses accuracy, precision, recall and F1-score to evaluate trained models. The F1-score was utilized for selecting the best model during hyperparameter tuning. The F1-score is the harmonic mean between precision and recall. This means that an increase in False Positives (affects Precision) or False Negatives (affects Recall) would penalise the F1-score.

I. Research Protocol

Firstly, we evaluate DYNAHUG using five (5) different tasks (**RQ1**) by training our classifer on the `text-classification`, `text-generation`, `feature-extraction`, `token-classification` and `image-classification` cluster. The goal is to measure the effectiveness of our approach on different clusters. Besides, we compare DYNAHUG to the current state-of-the-art (**RQ2**), we evaluate existing open-source detectors encompassing both static and dynamic analysis. We also evaluate whether LLMs can serve as an effective malicious PTM detector by replacing our classifier with an LLM and using the traces collected from dynamic analysis as an input. We excluded closed-source scanners and Restricted-Loading Environments (RLE) such as Weights-Only Unpickler [26] and PickleBall [25] because they do not scale to our evaluation setting (39K PTMs). Moreover, RLEs make incorrect assumptions about the user and could be easily bypassed. We have detailed our selection criteria and baseline setting in Section XI-C (Appendix) and Section XI-D (Appendix). Next, we conducted an ablation study (**RQ3**) examining our design choices. We examined the contribution of (a) dynamic analysis versus alternatives (i.e., static, dynamic or hybrid), (b) ML architecture (i.e., OCSVM, SGDSVM, and IF) and (c) feature set provided to the classifier for training and inference (i.e., *presence, frequency, sequence, process sequence*). Additionally, we investigate the performance of DYNAHUG with and without task clustering (**RQ4**). Finally, we performed sensitivity analysis (**RQ4**) of the classifier against varying training dataset size, i.e., top 1000, 2000 and 3000 repositories from HF. The settings for ablation, probing and sensitivity study is elaborated in Section XI-E (Appendix).

J. Additional Experiments

Probing and Sensitivity Study

Task Clustering: We found that DYNAHUG *with clustering outperforms* DYNAHUG *without clustering*. Table XII shows that DYNAHUG performs better when it is trained on individual task-based clusters (e.g., `text-generation`) versus on the Top-2000 models on HF (without clustering). This result holds

TABLE XI: DYNAHUG's sensitivity to varying dataset size (best results are in **bold** text)

Training Size	Detector	Benign Set	Malicious Set	Overall Performance						
				TP	TN	FP	FN	Precision	Recall	F1-score
1000	DYNAHUG	4/100	**100/100**	**100/100**	96/100	4/100	**0/100**	0.9615	**1.0000**	0.9804
2000	DYNAHUG (default)	4/200	198/200	198/200	196/200	4/200	2/200	0.9802	0.9900	0.9851
3000	DYNAHUG	**4/300**	297/300	297/300	**296/300**	**4/300**	3/300	**0.9867**	0.9900	**0.9884**

TABLE XII: Impact of clustering on DYNAHUG (best results are in **bold** text)

Cluster	Detector	Benign Set	Malicious Set	Overall Performance						
				TP	TN	FP	FN	Precision	Recall	F1-score
text-generation	DYNAHUG	4/200	198/200	198/200	196/200	4/200	2/200	0.9802	0.9900	0.9851
text-classification	DYNAHUG	7/200	199/200	199/200	193/200	7/200	1/200	0.9660	0.9950	0.9803
feature-extraction	DYNAHUG	5/200	197/200	197/200	195/200	5/200	3/200	0.9752	0.985	0.9801
token-classification	DYNAHUG	17/200	**200/200**	200/200	183/200	17/200	0/200	0.9216	1.0	0.9592
image-classification	DYNAHUG	**0/200**	**200/200**	**200/200**	**200/200**	**0/200**	**0/200**	**1.0**	**1.0**	**1.0**
non-clustered	DYNAHUG	9/200	199/200	199/200	191/200	9/200	1/200	0.9567	0.9950	0.9754

across all metrics. For instance, the F1-score of the DYNAHUG trained on the text generation cluster is 0.9851, while the non-clustered DYNAHUG has an F1-score of 0.9754. We attribute the performance of the clustered setting to the fact that it learns the specific behavior of a particular task better than the non-clustered model. This finding demonstrates the importance of clustering and motivates our design decision to cluster before training.

Task-based clustering contributes positively to DYNAHUG*'s performance.*

Sensitivity to training dataset size: We found that *the effectiveness of* DYNAHUG *slightly improves as the training dataset increases*. Table XI shows that training on a smaller dataset (1000) has a lower F1-score than training on the default DYNAHUG setting (2000), and training on even more PTMs (3000) outperforms the default DYNAHUG setting. DYNAHUG's performance is sensitive to the size of the dataset. We attribute the performance of larger datasets to the contribution of additional data points for generalizing the training. However, there is a trade-off between analysis cost and improved performance. Thus, we employ the 2,000 training data size for all other experiments, since the performance improvement with 3,000 PTMs is not significant (0.9884 vs. 0.9851) and it is computationally expensive to collect dynamic analysis on a larger set of models across each study, (e.g., thousands of models for additional settings in the ablation studies (RQ3)).

DYNAHUG*'s effectiveness slightly improves as the size of the training dataset increases.*

Advanced Attacks: We investigated the effectiveness of DYNAHUG and four (4) baseline scanners when faced with advanced malicious attacks. We focus on attacks that employ four (4) stealthy techniques, namely, (a) debugging and Anti-VM detection, (b) staged payloads, (c) obfuscated payloads and (d) delayed execution. We orchestrated all four attacks using the hard-to-detect PyPI library (slutterprime [83]), resulting in four advanced attack variants.

We found that DYNAHUG scales to these attacks, it detects these malicious PTMs containing such payloads since they still use system calls that are uncommon in benign PTMs. Among the four (4) SOTA scanners, only Fickling and ModelTracer detected these attacks. However, ModelTracer fails to detect the Anti-VM attack. Fickling detects the advanced malicious PTM because of the presence of the PyPI library import (which is not in its whitelist) and ModelTracer detects them because their traces still exhibit blacklisted system calls. ModelTracer does not detect the Anti-VM attack as it does not have `read()` and `openat()` as blacklisted syscalls (which are used to detect whether a VM is being used). DYNAHUG detects the Anti-VM attack because it recognises the additional `read()` and `openat()` calls. We note that even though Fickling and ModelTracer detected these attack, their detection methods lead to a lot of false positives or false negatives for unseen PTMs, due to their non-exhaustive rule lists. This result demonstrates the scalability of DYNAHUG to advanced attacks and the effectiveness of dynamic analysis in exposing such attacks. Our artifact [84] contains the malicious models resulting from this analysis, and our results.

DYNAHUG *scales to advanced attacks. It outperforms the SOTA scanners in detecting malicious PTMs invoking Anti-VM detection, as well as staged, obfuscated and delayed payloads.*